

Solving Airline Crew Scheduling Problems using Simulated Annealing and Genetic Algorithms

Nistha Kumar
Student ID: 2439590

Abstract

Airline crew scheduling is formulated as a Set Partitioning Problem (SPP), where each flight must be assigned to exactly one feasible crew pairing while minimising total operating cost. In this report, three metaheuristic approaches were implemented and compared: Simulated Annealing (SA), a standard Binary Genetic Algorithm (BGA), and an Improved Binary Genetic Algorithm (IBGA). The IBGA incorporates pseudo-random initialisation, a heuristic improvement operator, and stochastic ranking for constraint handling. Experimental results on three OR-Library benchmark instances show that IBGA consistently outperforms SA and BGA in terms of solution quality and robustness. In particular, IBGA achieves an average optimality gap of approximately 3–4% across all instances with 30 out of 30 feasible runs, demonstrating the effectiveness of specialised operators in improving performance on constrained combinatorial problems.

1 Introduction

Airline crew scheduling is a critical optimisation problem in the aviation industry, directly affecting operational efficiency and cost management. The problem can be formulated as a Set Partitioning Problem (SPP), where each flight (represented as a row) must be covered by exactly one crew pairing (represented as a column), and the objective is to minimise the total cost of the selected pairings. Since the SPP is NP-hard, exact optimisation methods become computationally expensive for larger instances, making metaheuristic approaches attractive practical alternatives.

The SPP can be formally defined as follows. Given a binary matrix $A \in \{0,1\}^{m \times n}$ where $a_{ij} = 1$ if column j covers row i , and a cost vector $c \in \mathbb{R}^n$, the objective is to find a binary vector $x \in \{0,1\}^n$ that:

$$\min \sum_{j=1}^n c_j x_j$$

subject to:

$$\sum_{j=1}^n a_{ij} x_j = 1, \quad \forall i = 1, \dots, m$$

where $x_j = 1$ indicates that column j (crew pairing) is selected. The equality constraint ensures that each row i (flight) is covered by exactly one selected column, enforcing both coverage (no uncovered flights) and exclusivity (no flight assigned to multiple crews).

This report investigates three metaheuristic algorithms for solving the airline crew scheduling problem. The first approach is Simulated Annealing (SA), a single-solution neighbourhood search method that explores the solution space using probabilistic acceptance of worsening moves. The second approach is a standard Binary Genetic Algorithm (BGA), which employs a population-based evolutionary framework combined with a penalty function to handle constraints. The third approach is an Improved Binary Genetic Algorithm (IBGA), based on the work of Chu and Beasley [2], incorporating pseudo-random initialisation and a heuristic improvement operator, together with the stochastic ranking method proposed by Runarsson and Yao [3] for constraint handling.

The algorithms were evaluated on three benchmark instances from the OR-Library [1], namely sppnw41, sppnw42, and sppnw43. For each algorithm and instance, 30 independent runs were conducted to assess average performance and robustness. The remainder of this report is organised as follows: Section 2 describes the three algorithms in detail, Section 3 outlines the parameter settings, Section 4 presents the computational results, and Section 5 provides a comparative discussion of the findings.

Table 1 summarises the characteristics of the three benchmark instances used in this study. Optimal values are taken from [2].

Table 1: Benchmark instance characteristics

Instance	Rows (m)	Columns (n)	Optimal
sppnw41	17	197	10972.50
sppnw42	23	1079	7485.00
sppnw43	18	1072	8897.00

2 Algorithms

2.1 Simulated Annealing

Simulated Annealing (SA) is a single-solution metaheuristic inspired by the cooling process in metallurgy. Starting from an initial solution, SA explores the neighbourhood by making small changes and accepting or rejecting them based on the Metropolis criterion (energy-based acceptance rule). Worse solutions can be accepted with a probability that decreases as the temperature cools, allowing the algorithm to escape local optima early in the search.

Constraints are handled using a penalty function $E(x) = cost(x) + \lambda \cdot violation(x)$, where $violation(x) = \sum_{i=1}^m |cover_i - 1|$ measures the deviation from the set partitioning constraint. Here, $cover_i$ is the number of selected columns that cover row i . A violation of zero indicates feasibility meaning every row is covered exactly once. Improvements ($\Delta E \leq 0$) are always accepted, while worsening moves are accepted with probability $e^{-\Delta E/T}$.

The neighbourhood structure adapts to the current state to handle the set partitioning constraints effectively. A pure random bit-flip neighbourhood was found to be insufficient for the larger instances (sppnw42, sppnw43), as the feasible region is sparse relative to the total search space. When the solution is infeasible, the algorithm targets violated rows by flipping a random covering column, directing the search toward feasibility rather than relying on random exploration. When feasible, a compound move is applied with probability 0.3: a randomly selected column is removed and the cheapest columns are greedily added to re-cover any uncovered rows. With probability 0.7, a simple flip removes a random selected column. The temperature is reduced geometrically according to $T_{k+1} = \alpha T_k$, with L evaluations performed at each temperature level.

Initialisation uses a greedy constructor that iteratively selects uncovered rows and adds the column with the best cost-to-coverage ratio, followed by pruning of redundant columns. This provides a feasible or near-feasible starting point, which is critical for the penalty-based search to function effectively. If no feasible solution is found during the search, a greedy fallback is applied, which repeatedly runs the greedy constructor with different seeds followed by pruning and repair until a feasible solution is obtained. Figure 1 illustrates the overall SA process.

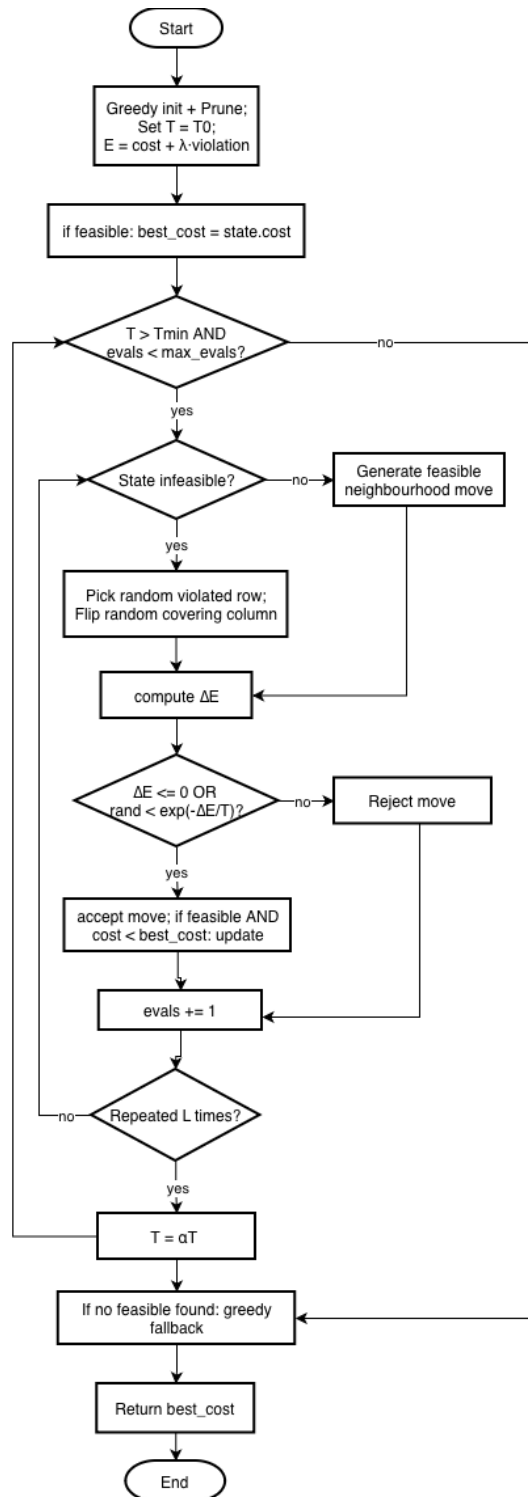


Figure 1: Flowchart for Simulated Annealing

Algorithm 1: Simulated Annealing

Input: instance, seed
Output: best feasible cost
initialise state using greedy constructor;
prune redundant columns;
if *state is feasible* **then**
 $\text{best_cost} \leftarrow \text{state.cost}$
set $T = T_0$, $E = \text{cost} + \lambda \cdot \text{violation}$;
while $T > T_{\min}$ and $\text{evals} < \text{max_evals}$ **do**
 for $i = 1$ to L **do**
 if *state is infeasible* **then**
 pick a random violated row;
 flip a random column covering that row;
 else
 if $\text{rand} < 0.3$ **then**
 remove a random selected column;
 greedily add cheapest columns to re-cover uncovered rows;
 else
 flip a random selected column;
 compute ΔE ;
 if $\Delta E \leq 0$ or $\text{rand} < e^{-\Delta E/T}$ **then**
 accept the move;
 if *feasible and cost* $< \text{best_cost}$ **then**
 update best_cost ;
 else
 reject the move;
 $\text{evals} \leftarrow \text{evals} + 1$;
 $T \leftarrow T \cdot \alpha$;
if *no feasible solution found* **then**
 apply greedy fallback;
return best_cost ;

2.2 Standard Binary Genetic Algorithm

The Binary Genetic Algorithm (BGA) is a population-based metaheuristic that evolves a set of candidate solutions through selection, crossover and mutation operators. Each solution is represented as a binary vector $x = (x_1, x_2, \dots, x_n)$, where $x_j = 1$ indicates that column j is selected. The algorithm maintains a population of individuals that improves over successive generations by favouring fitter individuals for reproduction.

Fitness is evaluated using a penalty function $E(x) = \text{cost}(x) + \lambda \cdot \text{violation}(x)$, where a higher penalty weight λ increases the selective pressure toward feasible solutions. Parent selection is performed using tournament selection: k individuals are sampled at random from the population and the one with the lowest penalty value is chosen. This process is repeated independently to select two parents for each offspring pair.

Offspring are generated using one-point crossover with probability p_c , where a random crossover point is chosen and the parent chromosomes are split and recombined. If crossover is not applied the offspring are copies of the parents. Each bit in the offspring is then independently flipped with probability p_m during mutation, introducing random variation to maintain diversity in the population.

After crossover and mutation, each child is pruned by removing columns whose covered rows are all covered by at least one other selected column. This reduces cost without affecting feasibility. If a child remains infeasible after pruning, a repair search is applied that iteratively adds the cheapest column

covering an uncovered row and removes the most expensive column from an over-covered row.

Elitism is used to preserve the top individuals across generations. At each generation, the population is sorted by penalty value and the best individuals are carried directly into the next generation without modification. The remaining slots are filled by the newly generated children.

The BGA implementation uses greedy initialisation rather than random bit strings, as random binary vectors are almost always highly infeasible for set partitioning instances and the penalty function alone cannot reliably guide the search to feasibility within the available generations. Redundant columns are pruned after each crossover operation to reduce cost without affecting coverage. A lightweight repair search is applied to infeasible children, iteratively adding the cheapest column to uncovered rows and removing the most expensive column from over-covered rows. These constraint-handling mechanisms ensure that all 30 runs produce feasible solutions across all instances. Figure 2 illustrates the overall BGA process.

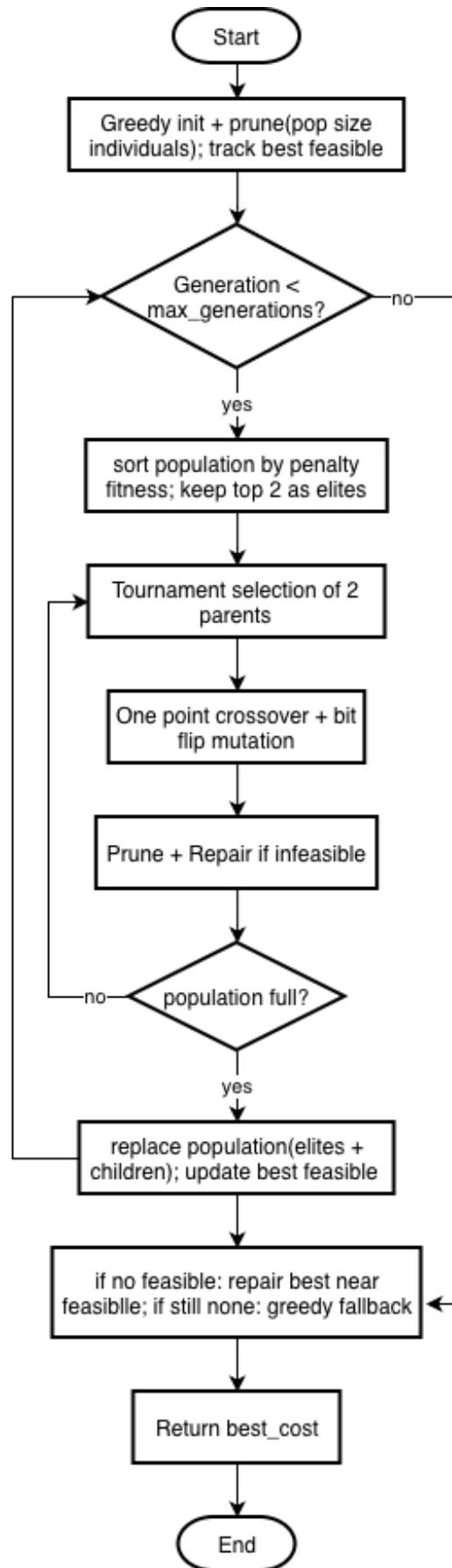


Figure 2: Flowchart for Standard BGA

Algorithm 2: Standard Binary Genetic Algorithm

Input: instance, seed
Output: best feasible cost
initialise population using greedy constructor + prune;
evaluate fitness using $f(x) = cost + \lambda \cdot violation$;
track best feasible cost and best near-feasible solution;
for $gen = 1$ **to** $max_generations$ **do**
 sort population by fitness;
 copy top elite individuals to next generation;
 while $children < pop_size - elite$ **do**
 select parents P1, P2 using tournament selection ($k = 3$);
 if $rand < p_c$ **then**
 | apply one-point crossover to get C1, C2;
 else
 | C1, C2 $\leftarrow$ copies of P1, P2;
 apply bit-flip mutation with probability p_m per bit;
 prune redundant columns from C1 and C2;
 if C1 *infeasible* **then**
 | apply repair search on C1
 if C2 *infeasible* **then**
 | apply repair search on C2
 add C1, C2 to children;
 population $\leftarrow$ elites + children;
 update best feasible cost and best near-feasible;
if *no feasible solution found* **then**
 repair best near-feasible solution;
 if *still no feasible solution* **then**
 | apply greedy fallback;
return best_cost;

2.3 Improved Binary Genetic Algorithm

The Improved Binary Genetic Algorithm (IBGA) extends the standard BGA with three key components from Chu and Beasley [2]: a pseudo-random initialisation method, a heuristic improvement operator and the stochastic ranking method from Runarsson and Yao [3] for constraint handling. These specialised operators are designed to exploit the structure of the Set Partitioning Problem, leading to significantly better solution quality compared to the standard BGA.

2.3.1 Pseudo-Random Initialisation

Unlike the greedy initialisation used in the standard BGA, the IBGA uses a pseudo-random constructor based on Algorithm 2 in [2]. Starting from an empty solution, the algorithm iteratively selects an uncovered row at random. With probability p_{random} , a random column covering that row is added. Otherwise, the column with the best cost-to-coverage ratio $cost_j / newly_j$ is selected, where $newly_j$ is the number of currently uncovered rows that column j would cover. This balance between random and greedy choices introduces diversity in the initial population while still producing reasonable starting solutions. After construction, redundant columns are pruned.

2.3.2 Heuristic Improvement Operator

The heuristic improvement operator, based on Algorithm 1 in [2], is a local search procedure applied to each infeasible child after crossover and mutation. It operates in three phases. First, if uncovered

rows exist, a column is added to cover a randomly chosen uncovered row, selecting the column that minimises $cost_j + \omega \cdot overlap_j$, where $overlap_j$ counts the number of already-covered rows that column j also covers and $\omega = 500$ is a weight that penalises overlap. This scoring penalises columns that introduce over-coverage. Second, redundant columns are pruned. Third, if over-covered rows remain, the most expensive column covering a randomly chosen over-covered row is removed. The operator repeats these phases iteratively, terminating early if the violation stops improving or the solution becomes feasible.

2.3.3 Stochastic Ranking

Constraint handling in the IBGA uses the stochastic ranking method proposed by Runarsson and Yao [3], replacing the penalty function approach used in the standard BGA. The population is sorted using a modified bubble sort over N sweeps. For each pair of adjacent individuals, with probability P_f the comparison is based on the objective function (cost), and with probability $1 - P_f$ it is based on constraint violation. When both individuals are feasible, the comparison is always based on cost. This probabilistic approach allows some infeasible solutions with low cost to survive in the population, maintaining diversity and avoiding premature convergence toward expensive feasible solutions. Stochastic ranking is used both for parent selection (selecting from the top half of the ranked population) and for survivor selection (ranking the combined parent and child populations and keeping the top individuals).

2.3.4 Evolution Process

The main evolutionary loop follows a similar structure to the standard BGA but with key differences. Parents are selected from the top half of the stochastic-ranked population rather than through tournament selection. One-point crossover is applied with probability p_c , followed by bit-flip mutation with probability p_m per bit. Each child is pruned, and if still infeasible, the improvement operator is applied. If the child remains infeasible after improvement, a repair search is used as a final fallback. Survivor selection combines the current population with the children, applies stochastic ranking to the combined pool, and retains the top pop_size individuals for the next generation. This combined $(\mu + \lambda)$ selection scheme ensures that good solutions from previous generations are not lost. Figure 3 illustrates the overall IBGA process.

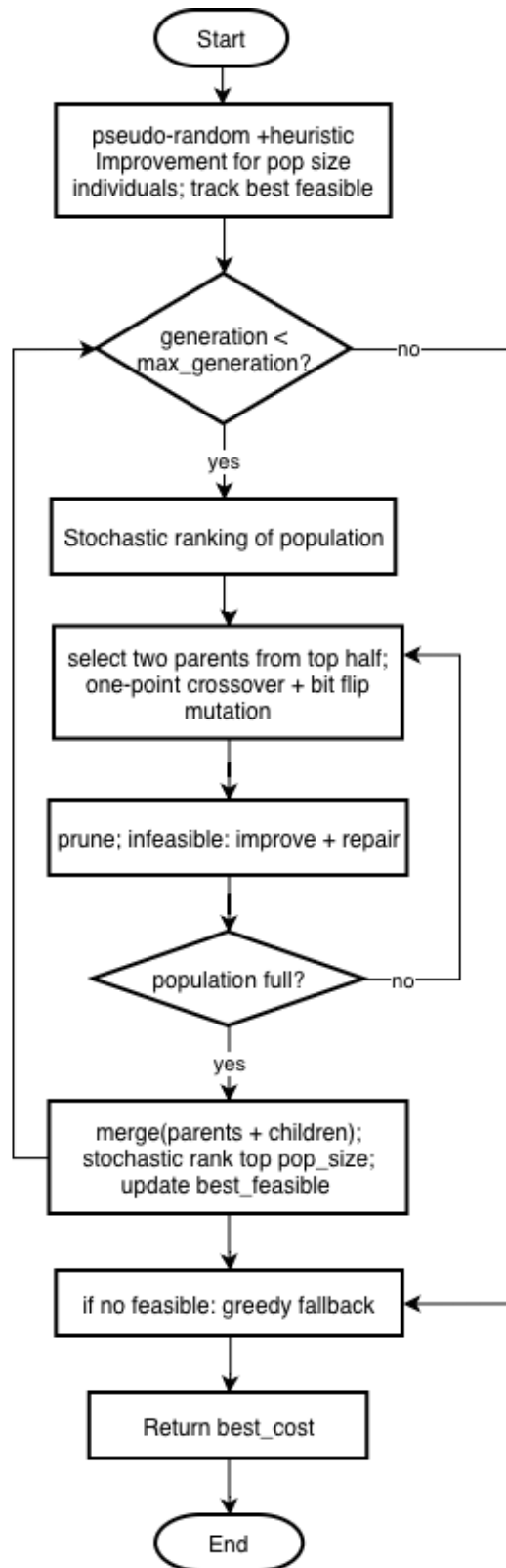


Figure 3: Flowchart for Improved BGA

Algorithm 3: Improved Binary Genetic Algorithm

Input: instance, seed
Output: best feasible cost
initialise population using pseudo-random constructor;
apply light improvement on each individual;
for $gen = 1$ **to** $max_generations$ **do**
 rank population using stochastic ranking;
 while $children < pop_size$ **do**
 select P1, P2 from top half of ranked population;
 if $rand < p_c$ **then**
 | apply one-point crossover to get C1, C2;
 else
 | C1, C2 $\leftarrow$ copies of P1, P2;
 apply bit-flip mutation with probability p_m per bit;
 prune redundant columns from C1 and C2;
 if $C1$ *infeasible* **then**
 | apply improvement operator on C1;
 | **if** $C1$ *still infeasible* **then**
 | apply repair search on C1;
 if $C2$ *infeasible* **then**
 | apply improvement operator on C2;
 | **if** $C2$ *still infeasible* **then**
 | apply repair search on C2;
 add C1, C2 to children;
 combine population and children;
 rank using stochastic ranking, keep top pop_size ;
 update best feasible cost;
if *no feasible solution found* **then**
 | apply greedy fallback;
return best_cost;

3 Parameter Settings

The parameter values for each algorithm were tuned experimentally and remain the same across all three problem instances. All algorithms were implemented in Python and executed on a MacBook with an M-series chip. Runtimes are wall-clock times for 30 runs on a single instance.

Table 2: Simulated Annealing Parameters

Parameter	Value
Penalty weight (λ)	1500
Initial temperature (T_0)	1500
Minimum temperature (T_{min})	0.001
Cooling rate (α)	0.9995
Chain length (L)	300
Max evaluations	500,000

Table 3: Standard BGA Parameters

Parameter	Value
Population size	60
Generations	300
Crossover rate (p_c)	0.9
Mutation rate (p_m)	0.005
Tournament size (k)	3
Elite count	2
Penalty weight (λ)	50,000

Table 4: Improved BGA Parameters

Parameter	Value
Population size	60
Generations	300
Crossover rate (p_c)	0.9
Mutation rate (p_m)	0.003
Stochastic ranking probability (P_f)	0.45
Improvement iterations	300
Initialisation steps	600
Initialisation randomness (p_{random})	0.2

4 Results

Each algorithm was run 30 times with independent seeds on each of the three problem instances. The optimal values reported in [2] are: sppnw41 = 10972.50, sppnw42 = 7485.00, sppnw43 = 8897.00.

The optimality gap for a single run is defined as:

$$\text{gap} = \frac{\text{cost} - \text{optimal}}{\text{optimal}} \times 100\%$$

where *cost* is the total cost of the best feasible solution found by that run, calculated as the sum of costs of all selected columns. In the results tables, Avg Cost is the arithmetic mean of the best feasible cost from each run, Std is the standard deviation and Best and Worst refer to the minimum and maximum feasible cost found across the 30 runs. The mean gap is the average of this value over all 30 independent runs, measuring how close an algorithm typically gets to the known optimum. A lower mean gap indicates better average solution quality, while a low standard deviation (Std) indicates consistent performance across runs.

Table 5: Results on sppnw41 (optimal: 10972.50)

Algorithm	Feasible	Avg Cost	Std	Best	Worst	Best Gap	Mean Gap
SA	30/30	11311.10	22.46	11307	11430	3.05%	3.09%
BGA	30/30	11322.00	45.77	11307	11457	3.05%	3.19%
IBGA	30/30	11311.10	22.46	11307	11430	3.05%	3.09%

Table 6: Results on sppnw42 (optimal: 7485.00)

Algorithm	Feasible	Avg Cost	Std	Best	Worst	Best Gap	Mean Gap
SA	30/30	10174.13	1260.30	7666	12288	2.42%	35.93%
BGA	30/30	9385.07	939.85	7656	10596	2.28%	25.38%
IBGA	30/30	7718.20	120.08	7656	7988	2.28%	3.12%

Table 7: Results on sppnw43 (optimal: 8897.00)

Algorithm	Feasible	Avg Cost	Std	Best	Worst	Best Gap	Mean Gap
SA	30/30	10821.60	587.58	9474	12218	6.49%	21.63%
BGA	30/30	10230.87	1011.08	8904	12980	0.08%	14.99%
IBGA	30/30	9250.67	316.06	8904	10036	0.08%	3.98%

Table 8 summarises the average performance across all three instances.

Table 8: Summary of average performance across all instances

Algorithm	Avg. Cost	Avg. Std	Best Single Gap	Feasibility
SA	10768.94	623.45	2.42%	90/90
BGA	10312.65	665.57	0.08%	90/90
IBGA	9426.66	152.87	0.08%	90/90

Table 9 reports the average wall-clock time per instance (total for 30 runs) for each algorithm.

Table 9: Average runtime across benchmark instances (seconds, 30 runs per instance)

Algorithm	Avg. Time (s)
SA	≈ 20
BGA	≈ 135
IBGA	≈ 190

Figure 4 shows sample terminal output for the sppnw42 instance, demonstrating the per-seed results and summary statistics produced by each algorithm. Similar output was generated for all three instances.

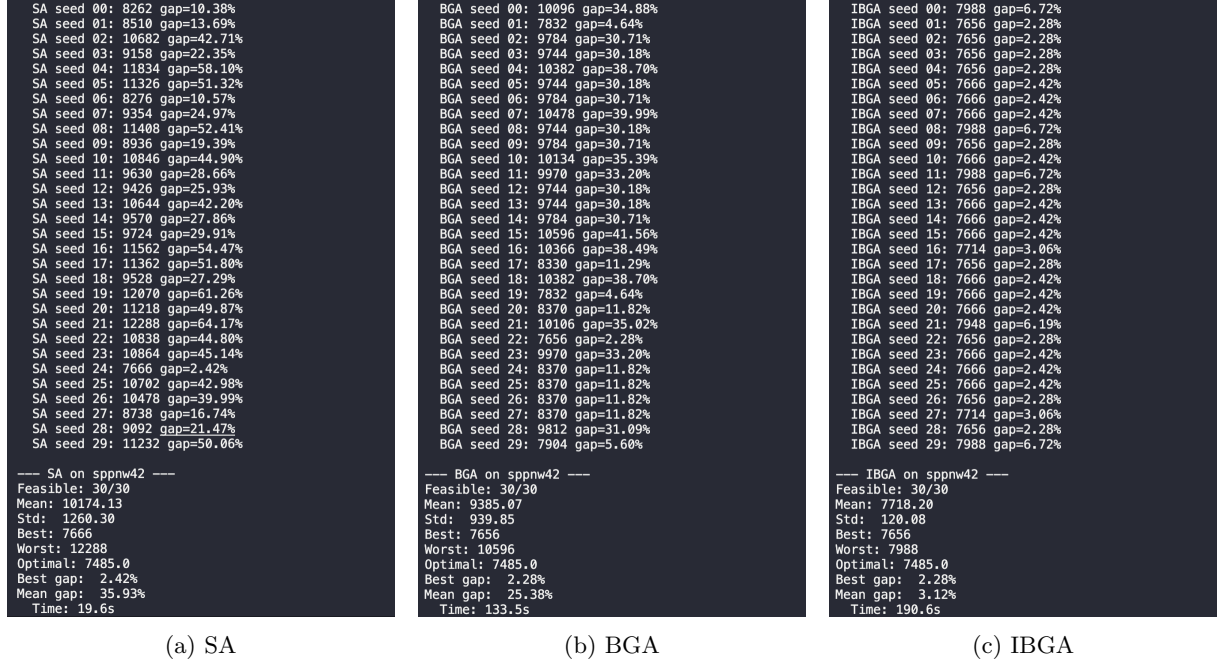


Figure 4: Terminal output for all three algorithms on sppnw42, showing per-seed costs and gaps, followed by summary statistics

5 Discussion

5.1 Comparison of Algorithms

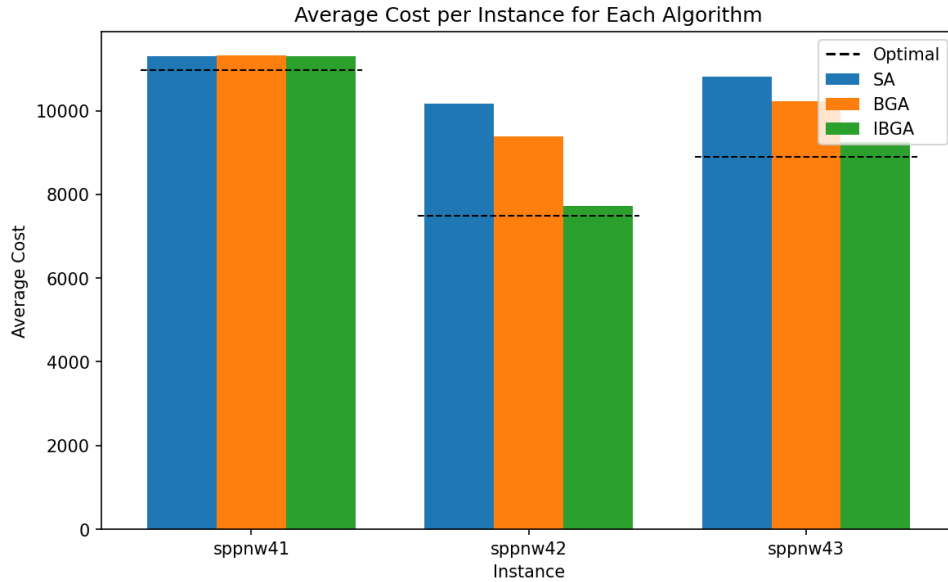


Figure 5: Average Cost Comparison Across Instances

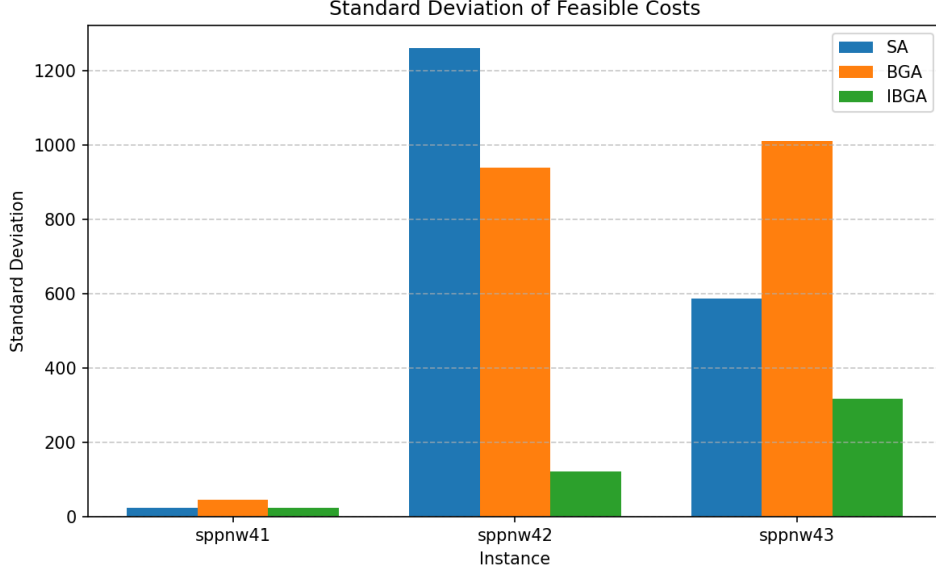


Figure 6: Standard deviation of feasible costs across all instances

As shown in Figures 5 and 6, the results clearly demonstrate the superiority of the Improved BGA. On sppnw41, all three algorithms perform similarly with mean gaps around 3%, suggesting that this is a relatively easy instance due to its small size (17 rows, 197 columns). The differences become significant on the larger instances sppnw42 and sppnw43. In fact, SA and IBGA produce identical summary statistics on this instance because both algorithms converge to the same local optimum (cost 11307) on 29 out of 30 runs, with only a single run reaching a slightly higher value. This confirms that sppnw41 has limited room for differentiation between methods.

On sppnw42, IBGA achieves a mean gap of 3.12% with a standard deviation of 120.08, while BGA and SA produce mean gaps of 25.38% and 35.93% respectively, with much higher standard deviations (939.85 and 1260.30). A similar pattern is observed on sppnw43, where IBGA maintains a mean gap of 3.98% compared to 14.99% for BGA and 21.63% for SA. This shows that IBGA not only finds better solutions but does so consistently across runs.

The improvement operator in IBGA is the primary driver of this performance difference. By iteratively covering uncovered rows and removing redundant or over-covered columns, it refines solutions in a problem-specific way that generic crossover and mutation cannot achieve. The pseudo-random initialisation also contributes by producing a diverse yet reasonable starting population, unlike the greedy initialisation in BGA which can lead to a homogeneous population trapped in similar local optima.

Stochastic ranking further improves IBGA by allowing some low-cost infeasible solutions to survive, maintaining population diversity. In contrast, the penalty function used by BGA and SA must balance a single λ value that either under-penalises violations (leading to infeasible solutions) or over-penalises them (pushing toward expensive feasible solutions).

SA performs worst on the harder instances despite being the fastest algorithm (approximately 20 seconds per instance). As a single-solution method, it lacks the population diversity that allows genetic algorithms to explore multiple regions of the search space simultaneously. However, its speed makes it suitable as a quick baseline or when computational resources are limited.

In terms of computational cost, SA is the fastest algorithm at approximately 20 seconds per instance, followed by BGA at around 135 seconds, and IBGA at approximately 190 seconds. The additional time required by IBGA is due to the improvement operator applied to each infeasible child, but this investment yields significantly better solution quality. Notably, on sppnw43, both BGA and IBGA achieve a best gap of just 0.08% from optimal, demonstrating that both algorithms are capable of finding near-optimal solutions on individual runs.

All three algorithms achieve 30/30 feasibility across all instances, confirming that the constraint handling mechanisms (penalty function for SA and BGA, stochastic ranking for IBGA) are effective.

5.2 Per-Instance Analysis

sppnw41 (17 rows, 197 columns): This is the easiest instance, with all three algorithms achieving mean gaps below 3.2%. The small problem size means the greedy constructor alone produces near-optimal solutions, leaving little room for the evolutionary operators to improve. The best solution found across all algorithms has cost 11307, which is 3.05% above optimal — suggesting that the gap may be due to the structure of the instance rather than algorithmic limitations.

sppnw42 (23 rows, 1079 columns): This is the hardest instance for SA and BGA, with mean gaps of 35.93% and 25.38% respectively. The large number of columns (1079) creates a vast search space where random or penalty-guided exploration is insufficient. IBGA reduces the mean gap to 3.12% with a standard deviation of just 120.08, demonstrating that the improvement operator is critical for navigating this search space. Notably, BGA and IBGA achieve identical best solutions (cost 7656, gap 2.28%), showing that BGA can occasionally find good solutions but lacks consistency.

sppnw43 (18 rows, 1072 columns): Results follow a similar pattern to sppnw42. BGA and IBGA both find a best solution of cost 8904, just 0.08% from optimal, demonstrating that near-optimal solutions are reachable. However, BGA’s mean gap of 14.99% and high standard deviation (1011.08) indicate unreliable performance, while IBGA maintains a mean gap of 3.98% with much lower variance (316.06). SA performs poorly with a mean gap of 21.63%, further highlighting the limitations of single-solution methods on larger instances.

5.3 Ranking Replacement vs Stochastic Ranking

The ranking replacement method in Chu and Beasley [2] and the stochastic ranking method in Runarsson and Yao [3] both aim to balance feasibility and solution quality when ranking individuals in a population.

The ranking replacement method in [2] uses a deterministic approach: feasible solutions are always ranked above infeasible ones, and within each group, individuals are sorted by objective value. This creates a strict boundary where no infeasible solution can outrank any feasible solution, regardless of how close it is to feasibility or how good its objective value is.

Stochastic ranking [3] uses a probabilistic approach instead. During each comparison in the bubble sort, with probability P_f the comparison is based on the objective function, and with probability $1 - P_f$ it is based on constraint violation. When both individuals are feasible, the comparison is always based on cost. This means that a low-cost infeasible solution can sometimes outrank an expensive feasible solution, depending on the random comparisons.

The key advantage of stochastic ranking is that it avoids the hard boundary between feasible and infeasible regions. By allowing near-feasible solutions with good objective values to survive, it maintains greater diversity in the population and can discover better feasible solutions through crossover of promising infeasible parents. The ranking replacement method is simpler and guarantees feasible solutions are preserved, but can lead to premature convergence as the population quickly becomes dominated by feasible but potentially suboptimal solutions.

5.4 Limitations and Future Work

Several limitations of this study should be noted. First, the three benchmark instances used are relatively small (17–23 rows, 197–1079 columns) compared to real-world airline scheduling problems, which can involve thousands of flights and millions of possible pairings. The performance differences between algorithms may change on larger instances.

Second, the SA and BGA parameters (particularly λ) were tuned manually through trial and error. A more systematic approach such as grid search or automated parameter tuning could potentially improve their performance, particularly on the harder instances where their gaps remain high.

Third, the SA and BGA algorithms struggle significantly on sppnw42 and sppnw43 (mean gaps of 15–36%), suggesting that the penalty function approach has fundamental limitations for constraint handling on these instances. The single λ value cannot adapt during the search, unlike stochastic ranking which dynamically balances feasibility and cost at every generation.

Future work could explore adaptive penalty weights that increase λ over time, alternative crossover operators such as uniform crossover, or hybrid approaches that combine SA with population-based methods. Additionally, testing on larger OR-Library instances would provide a more comprehensive evaluation of scalability.

6 Conclusion

This report implemented and compared three metaheuristic algorithms for solving the airline crew scheduling problem formulated as a Set Partitioning Problem. The Improved BGA, incorporating pseudo-random initialisation [2], a heuristic improvement operator [2], and stochastic ranking [3], consistently achieved mean gaps below 4% from the known optimal values across all three instances. The standard BGA and SA produced acceptable results on the smaller instance (sppnw41) but struggled on the larger instances (sppnw42, sppnw43), highlighting the importance of problem-specific operators in constrained combinatorial optimisation. All algorithms achieved 30/30 feasibility across all instances and runs.

References

- [1] John E Beasley. Or-library: distributing test problems by electronic mail. *Journal of the operational research society*, 41(11):1069–1072, 1990.
- [2] PC Chu and John E Beasley. Constraint handling in genetic algorithms: the set partitioning problem. *Journal of Heuristics*, 4(4):323–357, 1998.
- [3] Thomas P. Runarsson and Xin Yao. Stochastic ranking for constrained evolutionary optimization. *IEEE Transactions on evolutionary computation*, 4(3):284–294, 2000.